

FD04-EPIS-Informe Arquitectura de Software



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Informe de Arquitectura de Software

Sistema Analizador de Dependencias Multi-Lenguaje (DepAnalyzer)

Curso: Calidad y Pruebas de Software

Docente: Patrick Cuadros Quiroga

Integrantes:

Carbajal Vargas, Andre Alejandro (2023077287)

Yupa Gómez, Fátima Sofía (2023076618)

Tacna - Perú

2026

Sistema *Analizador de Dependencias Multi-Lenguaje (DepAnalyzer)*

Informe de Arquitectura de Software

Versión *1.1*

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-04-22	Versión inicial
1.1	ACV, FYG	ACV, FYG	P. Cuadros Q.	2026-06-23	Unificación del formato institucional

ÍNDICE GENERAL

1. Introducción
 1. Propósito (Diagrama 4+1)
 2. Alcance
 3. Definiciones, siglas y abreviaturas
 4. Organización del documento
2. Objetivos y restricciones arquitectónicas
 1. Priorización de requerimientos
 1. Requerimientos funcionales
 2. Requerimientos no funcionales
 2. Restricciones arquitectónicas
3. Representación de la arquitectura del sistema
 1. Vista de uso
 1. Diagrama de casos de uso
 2. Vista lógica
 1. Diagrama de sub-sistemas (paquetes)
 2. Diagrama de secuencia (vista de diseño)
 3. Diagrama de colaboración (vista de diseño)
 4. Diagrama de objetos
 5. Diagrama de clases
 6. Diagrama de base de datos
 3. Vista de implementación (vista de desarrollo)
 1. Diagrama de arquitectura de software
 2. Diagrama de arquitectura del sistema (diagrama de componentes)
 4. Vista de procesos
 1. Diagrama de procesos del sistema (diagrama de actividades)
 5. Vista de despliegue
 1. Diagrama de despliegue
4. Atributos de calidad del software
 1. Escenario de funcionalidad
 2. Escenario de usabilidad
 3. Escenario de confiabilidad

4. Escenario de rendimiento
5. Escenario de mantenibilidad
6. Otros escenarios de calidad
5. Evidencias de calidad y pruebas

1. Introducción

1.1 Propósito (Diagrama 4+1)

Este informe describe la arquitectura de software de *DepAnalyzer* siguiendo el enfoque 4+1, articulando las vistas de uso, lógica, implementación, procesos y despliegue, y vinculándolas con los requerimientos establecidos en FD03 y las restricciones de FD01/FD02.

Las decisiones de arquitectura priorizan:

- Cobertura funcional del análisis de dependencias multi-ecosistema (Maven, Gradle, npm y Python).
- Seguridad operativa en la actualización de versiones (confirmación explícita y backup).
- Interoperabilidad para CI/CD (salida JSON y códigos de salida).
- Mantenibilidad mediante separación modular del código.

1.2 Alcance

Este documento cubre la arquitectura del sistema implementado en la versión actual del proyecto, incluyendo:

- Flujo de análisis (analyze) y visualización interactiva (tui).
- Flujo de remediación guiada (update) con --dry-run y respaldo .bak.
- Integración con OSS Index y NVD mediante selección de fuente (--oss, --nvd o modo automático).
- Estructura de paquetes, componentes y comunicación interna.

No se incluye una arquitectura de aplicación web o móvil, ni persistencia basada en una base de datos relacional, porque el producto es una herramienta CLI/TUI local orientada a archivos y APIs externas.

1.3 Definiciones, siglas y abreviaturas

Término	Definición
API	Interfaz de programación de aplicaciones para comunicación entre sistemas.
CI/CD	Integración continua y entrega continua.
CLI	Interfaz de línea de comandos.
CVE	Identificador estándar de vulnerabilidad conocida.
CVSS	Métrica estándar para severidad de vulnerabilidades.
FD01	Informe de Factibilidad.
FD02	Informe de Visión.

Término	Definición
FD03	Informe de Especificación de Requerimientos.
JSON	Formato de intercambio de datos estructurados.
NVD	National Vulnerability Database de NIST.
OSS Index	Servicio de Sonatype para consulta de vulnerabilidades.
RF	Requerimiento funcional.
RNF	Requerimiento no funcional.
SCA	Software Composition Analysis.
TUI	Interfaz de usuario en terminal (modo pantalla completa).

1.4 Organización del documento

- Sección 2: presenta objetivos arquitectónicos y priorización de requerimientos.
- Sección 3: documenta las vistas arquitectónicas con diagramas Mermaid.
- Sección 4: define escenarios de atributos de calidad con criterios verificables.

2. Objetivos y restricciones arquitectónicas

2.1 Priorización de requerimientos

2.1.1 Requerimientos funcionales

ID	Descripción	Prioridad
RF-01	Detectar tipo de proyecto (pom.xml, build.gradle, build.gradle.kts)	Alta
RF-02	Parsear dependencias y repositorios por formato	Alta
RF-03	Consultar última versión en repositorios configurados	Alta
RF-04	Detectar CVEs con OSS Index	Alta
RF-05	Enriquecer CVEs con NVD de forma opcional	Media
RF-06	Clasificar vulnerabilidades directas y transitivas	Alta
RF-07	Renderizar salida legible en consola	Alta

ID	Descripción	Prioridad
RF-08	Exportar salida JSON estable	Alta
RF-09	Proveer interfaz TUI interactiva	Media
RF-10	Actualización guiada con confirmación y backup	Alta
RF-11	Simulación de actualización (--dry-run)	Media
RF-12	Fallar en CI por CVE crítico (--fail-on-critical)	Media

2.1.2 Requerimientos no funcionales

ID	Descripción	Prioridad
RNF-01	Portabilidad en Windows, Linux y macOS	Alta
RNF-02	Usabilidad técnica y ayuda clara en CLI	Alta
RNF-03	Confiabilidad ante fallas de red/APIs	Alta
RNF-04	Seguridad de credenciales de repositorios	Alta
RNF-05	Interoperabilidad mediante JSON y códigos de salida	Alta
RNF-06	Rendimiento razonable en proyectos medianos	Media
RNF-07	Mantenibilidad por arquitectura modular	Alta
RNF-08	Auditabilidad mediante CI/CD y pruebas	Media

2.2 Restricciones arquitectónicas

Restricción	Implicancia de diseño
JDK 25+ y Kotlin 2.3.10	La arquitectura se basa en JVM moderna y toolchain Gradle actual.
Herramienta local CLI/TUI	No se diseña backend web ni frontend web dedicado.
Dependencia de APIs externas (OSS Index/NVD)	Se requiere estrategia de degradación controlada ante errores o rate limit.
Sin base de datos relacional	Persistencia orientada a archivos (build, .bak, dependency-report.json).
Actualización no destructiva	Confirmación explícita y backup obligatorio antes de aplicar cambios.
Soporte multi-ecosistema (Maven/Gradle/npm/Python)	Se separan parsers y updaters por tipo de proyecto.
Integración CI	Se define salida JSON y modo fail-on-critical para automatización.

3. Representación de la arquitectura del sistema

3.1 Vista de uso

Esta vista muestra cómo los actores interactúan con las capacidades centrales del sistema.

3.1.1 Diagrama de casos de uso

```
flowchart LR
    U[Usuario técnico] --> UC1[Analizar proyecto]
    U --> UC2[Exportar reporte JSON]
    U --> UC3[Visualizar TUI]
    U --> UC4[Actualizar dependencias guiadas]
    U --> UC5[Simular actualizaciones]
    U --> UC6[Fallar build por CVE crítico]
    CI[Pipeline CI/CD] --> UC1
    CI --> UC2
    CI --> UC6
```

3.2 Vista lógica

La vista lógica refleja la descomposición del sistema en subsistemas y sus colaboraciones.

3.2.1 Diagrama de sub-sistemas (paquetes)

```
flowchart TD
    CLI[cli]
    CORE[core]
    PARSER[parser]
    REPO[repository]
    REPORT[report]
    UPDATE[update]
    TUI[tui]
    SECURITY[security]
    GRAPH[core.graph]
    CLI --> CORE
    CLI --> TUI
    CLI --> UPDATE
    CORE --> PARSER
    CORE --> REPO
    CORE --> REPORT
    CORE --> GRAPH
    UPDATE --> CORE
    UPDATE --> PARSER
```

UPDATE --> REPORT

REPO --> SECURITY

3.2.2 Diagrama de secuencia (vista de diseño)

```
sequenceDiagram
    actor Usuario
    participant CLI as DepAnalyzerCli
    participant PA as ProjectAnalyzer
    participant PD as ProjectDetector
    participant PR as Parser Maven/Gradle
    participant RC as RepositoryClient
    participant OI as OssIndexClient
    participant NVD as NvdClient
    participant RG as ReportGenerator
    Usuario ->> CLI: analyze . --nvd -o json
    CLI ->> PA: analyze(request)
    PA ->> PD: detect(projectPath)
    PD -->> PA: ProjectType
    PA ->> PR: parse dependencies/repositories
    PR -->> PA: dependencies + repositories
    PA ->> RC: getLatestVersion(...)
    RC -->> PA: latest versions
    PA ->> OI: getVulnerabilities(...)
    OI -->> PA: vulns OSS Index
    PA ->> NVD: getVulnerabilities(...)
    NVD -->> PA: vulns NVD
    PA -->> CLI: DependencyReport
    CLI ->> RG: toJson(report)
    RG -->> Usuario: dependency-report.json
```

3.2.3 Diagrama de colaboración (vista de diseño)

```
flowchart LR
    A[1 Usuario invoca comando] --> B[2 DepAnalyzerCli coordina flujo]
    B --> C[3 ProjectAnalyzer orquesta análisis]
    C --> D[4 ProjectDetector]
    C --> E[5 Parsers]
    C --> F[6 RepositoryClient]
    C --> G[7 OssIndexClient]
    C --> H[8 NvdClient opcional]
    C --> I[9 ReportGenerator/ConsoleRenderer]
    B --> J[10 UpdateCommand/UpdatePlanner]
    J --> K[11 BuildFileUpdater + BuildFileBackup]
```

3.2.4 Diagrama de objetos

```
classDiagram
    class cmd_Analyze {
        command = "analyze"
```



```

        useNvd = true
        output = "json"
    }

    class analyzer_ProjectAnalyzer {
        projectType = "GRADLE_KOTLIN"
        timeout = 1800
    }

    class report_DependencyReport {
        projectName = "demo-app"
        outdatedCount = 6
        vulnerableCount = 4
    }

    class updater_UpdatePlan {
        suggestions = 3
        dryRun = false
    }

    cmd_Analyze --> analyzer_ProjectAnalyzer: executes
    analyzer_ProjectAnalyzer --> report_DependencyReport: builds
    report_DependencyReport --> updater_UpdatePlan: optional remediation

```

3.2.5 Diagrama de clases

```

classDiagram
    class Depanalyzer
    class Analyze
    class Tui
    class Update

    class ProjectAnalyzer {
        +analyze(projectDir, ...): DependencyReport
    }

    class ProjectDetector {
        +detect(directory): ProjectType
    }

    class RepositoryClient {
        +getLatestVersion(repository, groupId, artifactId): String?
    }

    class OssIndexClient {
        +getVulnerabilities(dependencies): Map
    }

    class NvdClient {
        +getVulnerabilities(dependencies): Map
    }

```

```

class DependencyReport
class ReportGenerator
class ConsoleRenderer

class AnalyzerUpdatePlanner {
    +plan(projectDir, options): UpdatePlan
}

class BuildFileUpdater {
    <<interface>>
    +applyUpdate(file, suggestion): Boolean
}

class PomBuildFileUpdater
class GradleGroovyBuildFileUpdater
class GradleKotlinBuildFileUpdater
class BuildFileBackup

Analyze --> ProjectAnalyzer
Tui --> ProjectAnalyzer
Update --> AnalyzerUpdatePlanner
ProjectAnalyzer --> ProjectDetector
ProjectAnalyzer --> RepositoryClient
ProjectAnalyzer --> OssIndexClient
ProjectAnalyzer --> NvdClient
ProjectAnalyzer --> DependencyReport
ReportGenerator --> DependencyReport
ConsoleRenderer --> DependencyReport
AnalyzerUpdatePlanner --> BuildFileUpdater
BuildFileUpdater <|.. PomBuildFileUpdater
BuildFileUpdater <|.. GradleGroovyBuildFileUpdater
BuildFileUpdater <|.. GradleKotlinBuildFileUpdater
BuildFileBackup --> BuildFileUpdater

```

3.2.6 Diagrama de base de datos

El sistema no utiliza una base de datos relacional. El almacenamiento se realiza sobre archivos del proyecto analizado y salidas generadas por la herramienta.

flowchart LR

```

A[Proyecto Java] --> B[pom.xml / build.gradle / build.gradle.kts]
B --> C[Parsers y updaters]
C --> D[build file actualizado]
C --> E[backup .bak]
C --> F[dependency-report.json]

```

3.3 Vista de implementación (vista de desarrollo)

Describe cómo el diseño lógico se materializa en componentes de código y capas de responsabilidad.

3.3.1 Diagrama de arquitectura de software

flowchart TD

```
subgraph C1[Presentación]
```

```
  CLI[CLI Commands]
```

```
  TUI[TUI App]
```

```
end
```

```
subgraph C2[Aplicación]
```

```
  CORE[ProjectAnalyzer]
```

```
  UPD[UpdatePlanner/Updater]
```

```
end
```

```
subgraph C3[Dominio y Reporte]
```

```
  MODEL[DependencyReport / Vulnerability]
```

```
  RENDER[ConsoleRenderer / ReportGenerator]
```

```
end
```

```
subgraph C4[Infraestructura]
```

```
  PARSERS[Parsers Maven/Gradle]
```

```
  REPOS[RepositoryClient]
```

```
  OSS[OssIndexClient]
```

```
  NVD[NvdClient]
```

```
  SAFE[InputSafety]
```

```
end
```

```
CLI --> CORE
```

```
TUI --> CORE
```

```
CLI --> UPD
```

```
CORE --> MODEL
```

```
CORE --> RENDER
```

```
CORE --> PARSERS
```

```
CORE --> REPOS
```

```
CORE --> OSS
```

```
CORE --> NVD
```

```
REPOS --> SAFE
```

3.3.2 Diagrama de arquitectura del sistema (diagrama de componentes)

flowchart LR

```
C0[DepAnalyzerCli.kt] --> C1[ProjectAnalyzer.kt]
```

```
C0 --> C2[UpdateCommand.kt]
```

```
C0 --> C3[AnalyzeTuiApp.kt]
```

```
C1 --> C4[ProjectDetector.kt]
```

```
C1 --> C5[PomDependencyParser.kt]
```

```

C1 --> C6[GradleGroovyDependencyParser.kt]
C1 --> C7[GradleKotlinDependencyParser.kt]
C1 --> C8[RepositoryClient.kt]
C1 --> C9[OssIndexClient.kt]
C1 --> C10[NvdClient.kt]
C1 --> C11[ReportGenerator.kt]
C1 --> C12[ConsoleRenderer.kt]
C2 --> C13[UpdatePlanner.kt]
C2 --> C14[BuildFileUpdater.kt]
C2 --> C15[BuildFileBackup.kt]
C8 --> C16[InputSafety.kt]

```

3.4 Vista de procesos

Modela la coordinación de tareas en ejecución, incluyendo escenarios de degradación por fallas externas.

3.4.1 Diagrama de procesos del sistema (diagrama de actividades)

flowchart TD

```

A([Inicio]) --> B[Recibir comando CLI/TUI]
B --> C[Detectar tipo de proyecto]
C --> D[Parsear dependencias y repositorios]
D --> E[Consultar versiones recientes]
E --> F[Consultar CVEs OSS Index]
F --> G{--nvd}
G -- Sí --> H[Consultar NVD y fusionar resultados]
G -- No --> I[Continuar]
H --> J[Construir DependencyReport]
I --> J
J --> K{output json}
K -- Sí --> L[Generar dependency-report.json]
K -- No --> M[Renderizar consola/TUI]
L --> N{usuario ejecuta update}
M --> N
N -- Sí --> O[Planificar sugerencias]
N -- No --> P{confirmación usuario}
O --> P
P -- Sí --> Q[Crear backup y aplicar cambios]
P -- No --> R[Registrar omitidas]
Q --> S([Fin])
R --> S

```

3.5 Vista de despliegue

Presenta escenarios de ejecución física del sistema en local y en CI.

3.5.1 Diagrama de despliegue

```
flowchart TD
    subgraph Local[Entorno Local Desarrollador]
        U[Usuario]
        BIN[depanalyzer / depanalyzer.bat / depanalyzer.exe]
        JDK[JDK 25+]
        PROJ[Proyecto Java objetivo]
        OUT[dependency-report.json / .bak]
    end

    subgraph External[Servicios Externos]
        OSS[OSS Index API]
        NVD[NVD API]
        REPO[Maven/Repositorios remotos]
    end

    subgraph CI[GitHub Actions]
        WF[Workflow test.yml]
        REL[Workflow release-native.yml]
    end

    U --> BIN
    BIN --> JDK
    BIN --> PROJ
    BIN --> OUT
    BIN --> OSS
    BIN --> NVD
    BIN --> REPO
    WF --> BIN
    REL --> BIN
```

4. Atributos de calidad del software

4.1 Escenario de funcionalidad

Elemento	Definición
Fuente de estímulo	Usuario técnico o pipeline CI
Estímulo	Ejecutar análisis sobre un proyecto Maven, Gradle, npm o Python
Entorno	CLI local o runner CI con conectividad de red
Respuesta	Detectar tipo de proyecto, listar desactualizadas y CVEs, emitir reporte
Medida de respuesta	Cobertura funcional alineada con RF priorizados de FD03

4.2 Escenario de usabilidad

Elemento	Definición
Fuente de estímulo	Usuario técnico básico/intermedio
Estímulo	Necesita interpretar hallazgos y tomar decisiones de actualización
Entorno	Terminal local con o sin soporte de color
Respuesta	CLI con ayuda consistente, salida legible, modo TUI y opción --no-color
Medida de respuesta	Comprensión operativa de salida $\geq 80\%$ en evaluación interna (FD02)

4.3 Escenario de confiabilidad

Elemento	Definición
Fuente de estímulo	Fallo de red, error de API externa o rate limiting
Estímulo	Error HTTP/timeout en OSS Index o NVD
Entorno	Ejecución normal de análisis
Respuesta	Degradación controlada: continuar análisis con información disponible y advertencia
Medida de respuesta	Ejecución no aborta por falla parcial externa en escenarios manejables

4.4 Escenario de rendimiento

Elemento	Definición
Fuente de estímulo	Usuario ejecuta análisis en proyecto mediano
Estímulo	Consulta de dependencias, versiones y CVEs
Entorno	Equipo local de referencia (FD01)
Respuesta	Completar análisis con tiempos operables para ciclo de desarrollo
Medida de respuesta	Objetivo referencial ≤ 30 s en caso pequeño/mediano (FD02/FD03)

4.5 Escenario de mantenibilidad

Elemento	Definición
Fuente de estímulo	Equipo de desarrollo requiere agregar parser o nueva fuente de vulnerabilidad
Estímulo	Cambio evolutivo en módulo específico

Elemento	Definición
Entorno	Código Kotlin modular con pruebas
Respuesta	Modificación acotada por paquete sin impacto transversal no controlado
Medida de respuesta	Bajo acoplamiento entre parser, repository, core, report, update

4.6 Otros escenarios de calidad

4.6.1 Seguridad

Elemento	Definición
Fuente de estímulo	Proyecto con repositorios privados y credenciales
Estímulo	Consulta de metadatos de versión con autenticación
Entorno	Ejecución local con variable de confianza de hosts
Respuesta	Enviar credenciales solo a hosts HTTPS permitidos por allowlist
Medida de respuesta	Cumplimiento de política DEPANALYZER_TRUSTED_CREDENTIAL_HOSTS

4.6.2 Interoperabilidad

Elemento	Definición
Fuente de estímulo	Pipeline CI/CD
Estímulo	Consumir resultado de análisis en etapa automatizada
Entorno	GitHub Actions u otro motor CI
Respuesta	Salida JSON parseable y --fail-on-critical para control de estado
Medida de respuesta	Integración estable de reporte y política de fallo por criticidad

5. Evidencias de calidad y pruebas

Los reportes se publican en GitHub Pages para centralizar la evidencia solicitada por el curso. La URL base del sitio es:

<https://upt-faing-epis.github.io/proyecto-si784-2026-i-u2-analizador-de-dependencias-2/>

Evidencia	Enlace público	Fuente / herramienta
Sonar	Reporte Sonar	SonarCloud / Gradle Sonar

Evidencia	Enlace público	Fuente / herramienta
Semgrep	Reporte Semgrep	Semgrep CLI
Snyk	Reporte Snyk	Snyk CLI
Pruebas unitarias	Reporte unitario	Gradle Test / JUnit 5
Pruebas de integración	Reporte de integración	Tests bajo src/test/kotlin/com/depanalyzer/integration
Pruebas de mutación	Reporte de mutación	PIT
Pruebas de interfaz	Reporte de interfaz	Tests CLI/TUI
Pruebas BDD	Reporte BDD	Escenarios Gherkin/Markdown de uso CLI
Documentación autogenerada	API Docs	Dokka

Los reportes de Sonar y Snyk dependen de los secretos SONAR_TOKEN y SNYK_TOKEN configurados en GitHub. Si esos secretos no están disponibles, el workflow publica una página de estado indicando la configuración pendiente sin bloquear la publicación del resto de evidencias.

6. Ingeniería Inversa e Infraestructura

Los diagramas se contrastaron con src/main/kotlin, build.gradle.kts, .github/workflows e infrastructure/terraform; representan componentes implementados.

flowchart TD

```

REPO[Repositorio] --> CI[CI]
REPO --> QUALITY[Quality and Pages]
REPO --> RELEASE[Native Release]
REPO --> TF[Terraform]
QUALITY --> JACOCO[JaCoCo >= 70%]
QUALITY --> PIT[PIT]
QUALITY --> BDD[Cucumber]
QUALITY --> UI[Playwright + videos]
QUALITY --> STATIC[Sonar + Semgrep + Snyk]
QUALITY --> PAGES[GitHub Pages]
RELEASE --> BIN[Binarios multiplataforma]
TF --> ENV[Environment github-pages]
TF --> VARS[Variables de calidad]

```

Vista	Fuente de ingeniería inversa
Casos de uso	Comandos analyze, tui, update

Vista	Fuente de ingeniería inversa
Secuencia	ProjectAnalyzer, parsers, OSS/NVD y reportes
Clases	Data classes y servicios Kotlin
Componentes	Paquetes de com.depanalyzer
Despliegue	Actions, GraalVM, Releases, Pages y JReleaser
Infraestructura	Recursos Terraform del proveedor GitHub